

scene_objects 를 학습 입력(observation)에 넣을 것인가

작성 2026-08-23 · 결정 대기 (export 단계 결정)

적재는 이미 결정·구현되었다 — /scene/objects 는 rosbag2 에 녹화되고 scene_objects 테이블에 적재된다. 본 문서가 다루는 것은 그 다음 층, 즉 **export 시 observation 컬럼에 포함할 것인가**뿐이다. 두 층을 섞으면 논의가 엉킨다(§1).

배경: [auto_episode_collection_draft.md](#) §5 가 "학습 입력으로 넣는 것은 별도 판단이 필요하다"로 남겨둔 항목이다.

1. 질문을 정확히 놓기 — 두 층은 다른 결정이다

층	질문	상태	되돌릴 수 있나
적재	rosbag/DB 에 남길 것인가	☒ 남긴다 (2026-08-23 구현)	☒ 안 남기면 그 에피소드는 영구 손실
관측	정책의 입력 벡터에 넣을 것인가	☐ 본 문서의 주제	☒ export 는 읽기 전용 소비자라 언제든지 번복

적재를 먼저 확정 한 이유는 비대칭 때문이다. 관측 포함 여부는 [framework_design §5.6](#)이 보장하듯 **export 시점에 컬럼 매핑으로 고르는 문제**라 나중에 바뀌도 비용이 없다. 반면 적재하지 않은 에피소드는 사후에 ground truth 를 복원할 방법이 없다.

그래서 남은 결정은 "**이미 있는 컬럼을 정책에 먹일 것인가**" 하나다.

2. 무엇을 넣는다는 것인가

scene_objects 한 행은 다음과 같다.

```
[{"name": "cube_0", "type": "box", "dimensions": [0.05, 0.05, 0.05],
  "position": [0.4, 0.1, 0.025], "orientation": [0.0, 0.0, 0.0, 1.0]}
```

observation 으로 쓰려면 이 가변 길이 구조를 고정 길이 벡터로 눌러야 한다. 이것이 단순한 컬럼 추가가 아닌 이유다 — 물체 수가 에피소드마다 다르고, dimensions 길이가 종류마다 다르며(box 3 / sphere 1 / cylinder 2), 배열 순서는 계약상 보장되지 않는다 (name 으로 지목한다). 인코딩 선택은 §6 에 있다.

3. 선택지

A. 넣지 않는다 — 카메라만 관측 (문서의 잠정 입장)

정책 입력은 joint_states · ee_pose · 카메라 이미지. 물체 위치는 픽셀에서 정책이 스스로 추론한다.

근거	시뮬레이터의 ground-truth object pose 는 실기에 존재하지 않는다. 그대로 관측에 넣으면 실기 이관 시 입력 자체를 만들 수 없어 sim 에서만 도는 정책이 된다
전제	카메라가 물체를 실제로 담고 있어야 한다. 이 전제가 깨지면 §4 의 실패로 직행
비용	픽셀에서 pose 를 짜내야 하므로 표본 효율이 낮고 수렴이 느리다
부가 이득	학습된 정책이 그대로 실기 후보가 된다

B. 넣는다 — ground truth 를 직접 관측으로

근거	가장 빨리, 가장 높은 성공률로 수렴한다. 배선·데이터 경로 검증에는 이 편이 낫다
한계	실기 이관 불가. 같은 정책을 실기에 올리려면 pose estimator 를 붙여 입력을 합성해야 하고, 그 순간 추정 오차가 학습 때 없던 분포 이동으로 들어온다
용도	baseline / upper bound. "데이터가 문제인가 표현이 문제인가"를 가르는 대조군

C. 절충 — 학습에만 쓰고 추론에는 안 쓴다

ground truth 를 정책 입력이 아닌 다른 자리에 넣는 방식들이다. 어느 것도 추론 시점의 입력 스펙을 바꾸지 않으므로 A 의 실기 이관성을 유지한다.

방식	어디에 쓰나
보조 손실 (auxiliary loss)	정책이 이미지에서 물체 pose 를 예측하도록 겹가지 head 를 달고, GT 로 지도한다. 표현 학습을 가속하되 추론 시 그 head 는 버린다
teacher-student 종류	GT 를 보는 teacher 를 먼저 학습(=B)시키고, 카메라만 보는 student 가 그 행동을 모방한다. teacher 가 상한을 알려 주므로 student 의 실패가 "표현 문제"인지 "데이터 문제"인지 갈린다
pose estimator 별도 학습	(이미지, GT pose) 쌍으로 estimator 를 따로 학습하고, 정책은 그 출력만 본다. 실기에서는 estimator 를 실기 데이터로 재학습·보정

C 는 A 와 배타적이지 않다 — **A 의 학습 파이프라인 위에 얹는 층**이다.

4. A 가 실패하는 조건 — 이것이 판단의 핵심

A 는 무조건 안전한 선택이 아니다. **물체 위치가 어떤 관측 채널에도 담기지 않으면**, 매 회 랜덤화한 위치가 곧 관측 불가능한 잠재변수가 된다.

수집 목표가 §0 의 "물체 위치를 매 회 랜덤화"이므로, 이 조건은 부수적이지 않고 정면으로 걸린다.

에피소드 1: 관측(관절각·EE pose) $\approx X \rightarrow$ 정답 행동: 왼쪽으로
에피소드 2: 관측(관절각·EE pose) $\approx X \rightarrow$ 정답 행동: 오른쪽으로

↓

회귀 손실이 충돌을 평균으로 해소 $\rightarrow$ 팔이 가운데로 간다

같은 관측에 서로 다른 정답이 붙으면 behavior cloning 은 그 다중성을 **평균**으로 없앤다. 데이터를 늘려도 평균이 정확해질 뿐 문제는 풀리지 않는다. 이것이 부분 관측 $\rightarrow$ 행동 다중성 $\rightarrow$ mode averaging 의 전형적 경로다.

따라서 **A 의 성립 여부는 카메라 커버리지에 전적으로 달려 있다**. FOV·가림·시점 수가 부족해 물체가 안 보이면, "A 를 선택했다"가 아니라 "관측을 잃었다"가 된다.

5. 현재 스택의 제약 — 지금 A 를 적용하면

세 가지가 A 의 전제를 만족시키지 못한다. 셋 다 **한시적**이라는 점이 중요하다.

제약	현재	해소 시점
썸 카메라 부재	image_pipeline.yaml 의 <code>camera.id</code> 가 mp4 파일이다 — 실 카메라가 연결되지 않아 쓰는 임시 소스이며 썸과 무관한 영상이다	실 카메라 연결 시
mock 에 물리 없음	물체가 밀리지도 떨어지지도 않아 <code>scene_objects</code> 가 에피소드 내내 초기 배치 그대로다. 관측으로서의 정보량이 사실상 초기값 한 장	Gazebo 백엔드 연동 (§6 작업 6)
단일 시점	카메라 1대. 가림에 취약하다	필요 시 시점 추가

즉 **지금 이 결정을 확정하는 것은 이르다**. 카메라가 붙기 전까지는 A 도 B 도 실질적으로 검증할 수 없고, 그 사이에 필요한 것은 (이미 한) 적재뿐이다.

6. 권고

적재 O / observation X 를 기본값으로 두고, 카메라가 붙은 뒤 A 를 실측한다.

`scene_objects` 는 그때까지 라벨·큐레이션 용도로만 쓴다.

1. **지금** — 적재만 한다(완료). 관측 컬럼 매핑에는 넣지 않는다.
2. **실 카메라 연결 후** — 카메라가 물체를 담는지부터 확인한다. GT pose 를 이미지에 투영해 눈으로 대조하는 것이 가장 빠르다.
3. **A 를 학습시켜 본다**. 실패하면 그 원인을 §4 의 mode averaging 인지 표현 학습 부족인지 갈라야 하는데, **B 를 대조군으로 돌리는 것이 그 판별법**이다. B 가 잘 되고 A 가 안 되면 관측에 정보가 없다는 뜻이고, 둘 다 안 되면 데이터·행동 정의 문제다.
4. **A 가 표현 학습 때문에 느린 것으로 판명되면** C(보조 손실 → teacher-student)를 엮는다.
이때도 추론 입력 스펙은 A 그대로다.

B 를 최종안으로 삼지 않는다. 배선 검증과 대조군으로는 유용하지만, 그것으로 만든 정책은 실기에 올릴 수 없다.

재검토 트리거

- 실 카메라가 연결되어 썸을 담기 시작할 때 → §5 의 첫 제약이 사라지므로 A 를 실측한다
- Gazebo 백엔드가 붙어 물체가 물리적으로 움직일 때 → `scene_objects` 가 비로소 시계열 정보를 갖는다
- 카메라가 있는데도 A 가 안 될 때 → C 로 올라간다

7. 넣기로 할 경우의 실무 쟁점

결정이 B/C 로 가면 아래가 따라온다. **지금 풀 필요는 없고, 여기 적어 두어 그때 다시 설계하지 않게 한다.**

- **가변 길이 인코딩** — 물체 수가 다르다. 슬롯 고정(최대 N개 + 마스크), 이름 기준 정렬 후 절단, set-transformer 류 순열 불변 인코더 중 택일. 배열 순서는 계약상 보장되지 않으므로 **인덱스를 그대로 쓰면 안 된다** — name 으로 정렬·지목한다.
- **좌표계** — frame_id 를 함께 저장하는 이유가 여기 있다. 백엔드가 world→base 변환을 빠뜨려도 값은 그럴듯하므로, 관측으로 쓰기 전 panda_link0 인지 반드시 확인한다.
- **쿼터니언** — DB 에는 ROS 규약 xyzw 로 들어간다. wxyz 로 읽으면 unit norm 을 그대로 통과하면서 '그렇듯하게 틀린 자세'가 된다.
- **dimensions 순서** — shape_msgs/SolidPrimitive 를 따른다. cylinder 는 **높이·반지름** 순으로 직관과 반대다.
- **정규화** — position 은 m 단위 raw 값이다. 관절각과 스케일이 다르므로 표준화가 필요하다.

8. 데이터 경로 (참고)

관측에 넣든 안 넣든 아래는 이미 동작한다.

scene 상태 노드 (2 Hz) → /scene/objects → rosbag2 → scene_objects 테이블
(백엔드마다 하나) TRANSIENT_LOCAL episode_id 로 분할

구성요소	위치
발행 (mock)	mock_scene_state_node.py
메시지 정의	src/rdfp_msgs/msg/SceneObjects.msg · SceneObject.msg
녹화 대상 등록	config/recording_topics.list
적재	writers/scene_objects.py · readers/scene_objects.py
테이블	sql/schema.sql 의 scene_objects

자동 라벨링(place 성공 판정)은 에피소드 마지막 행을 읽어 물체별로 푼다.

```
SELECT o->>'name' AS name, (o->'position'->>0)::float8 AS x
FROM scene_objects s, jsonb_array_elements(s.objects) o
WHERE s.episode_id = %s AND s.object_count > 0
ORDER BY s.stamp_ts DESC;
```

9. 참고

- [auto_episode_collection_draft.md](#) — §5 가
이 질문을 남긴 곳, §2 가 /scene/objects 계약, §6 이 작업 순서
- [rdfp_framework_design.md](#) — §2.2 action/observation 구분,
§5.6 export 가 읽기 전용 소비자라는 근거
- [데이터셋 후처리기 설계서.md](#) — 적재 파이프라인 전반